INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Servicio en segundo plano para sincronizar datos en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	10	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	03/11/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Mg Roxana Evelyn Limache Calatayud					

/ex

SOLUCIÓN Y RESULTADOS

- Link Repositorio: https://github.com/shanccom/IDNP_Repo/tree/main/lab10SincronizacionServicioCompose

I. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para implementar un Service que simula la sincronización de la lista de edificios y avisa a la UI mediante un BroadcastReceiver:

Paso 1. Crear un nuevo proyecto

Crear un nuevo proyecto. nombre sugerido para el proyecto: “SincronizacionServicio_Compose”, Package name: com.example.sincronizacionservicio_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

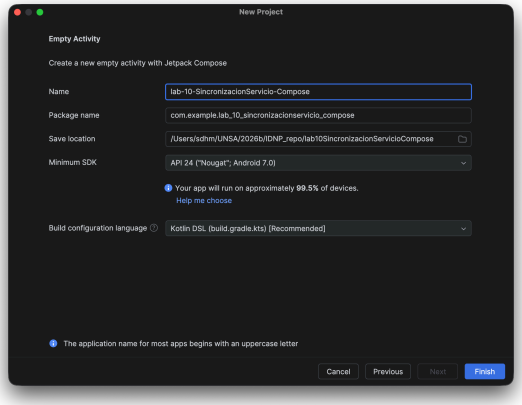


Figura 1: Paso 1: Creación del proyecto SincronizacionServicio_Compose en Android Studio con Kotlin y el SDK mínimo de la plantilla.

Paso 2. Abrir el archivo AndroidManifest.xml y declarar el nuevo Service

Abrir el archivo AndroidManifest.xml y declarar el nuevo Service dentro de la etiqueta `<application>`:

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    <application
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="lab-10-SincronizacionServicio-Compose"
            android:theme="@style/Theme.Lab10SincronizacionServicioCompose"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <service
            android:name=".SincronizacionService"
            android:exported="false" />

    </application>

</manifest>
```

Figura 2: Código (AndroidManifest.xml – fragmento dentro de `<application>`)

Paso 3. Agregar importaciones

Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

```kotlin
package com.example.lab_10_sincronizacionservicio_compose

import android.app.Service
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.Bundle
import android.os.IBinder
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
```

Figura 3: Código (importaciones)

Paso 4. Crear la clase SincronizacionService

Crear la clase SincronizacionService, heredando de Service, y sobrescribir onBind() devolviendo null, ya que no será un Bound Service. Por ahora dejamos pendiente la lógica de sincronización:

```kotlin
2 Usages
class SincronizacionService : Service() {

    override fun onBind(intent: Intent?): IBinder? = null
```

Figura 4: Código (SincronizacionService – Paso 4)

Paso 5. Sobrescribir onStartCommand()

Sobrescribir onStartCommand(), lanzando una coroutine en Dispatchers.IO que simule una tarea de sincronización con delay(3000) y calcule un resultado simulado (por ejemplo, un número aleatorio de edificios actualizados):



Figura 5: Código (SincronizacionService – Paso 5)

Paso 6. Construir el Intent y enviar el Broadcast

Al terminar la coroutine, construir un Intent con una acción propia, indicar con setPackage(packageName) que el aviso es solo para esta aplicación, agregar el resultado como extra, enviarlo con sendBroadcast(), registrar un mensaje en el Logcat y llamar a stopSelf() para finalizar el Service. Con esto, SincronizacionService queda completo:



Figura 6: Código (SincronizacionService completo)

Paso 7. Crear el Composable PantallaSincronizacion

Crear el Composable PantallaSincronizacion, obteniendo el contexto actual y una variable de estado para el resultado recibido. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:

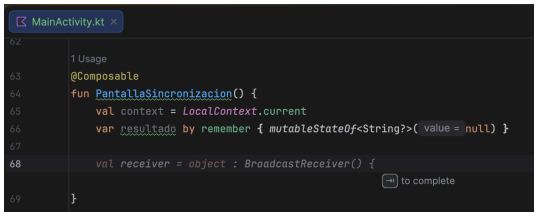


Figura 7: Código (PantallaSincronizacion – Paso 7)

Paso 8. Registrar el BroadcastReceiver con DisposableEffect

Dentro de un DisposableEffect(Unit), crear un BroadcastReceiver que en onReceive() registre un mensaje en el Logcat y actualice resultado con el extra recibido, registrarlo con ContextCompat.registerReceiver() indicando RECEIVER_NOT_EXPORTED, y desregistrarlo en onDispose { }:

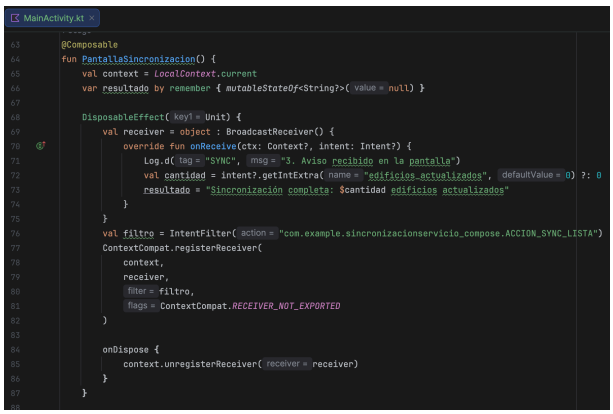


Figura 8: Código (PantallaSincronizacion – Paso 8)

Paso 9. Agregar botón de sincronización e indicador de progreso

Agregar un botón «Sincronizar» que inicie el Service con startService(), y un indicador de progreso que se muestre mientras resultado sea null. El Column usa statusBarsPadding() para que el contenido no quede debajo de la barra de estado. Con esto, PantallaSincronizacion queda completo:



Figura 9: Código (PantallaSincronizacion completo)

Paso 10. Integrar PantallaSincronizacion en MainActivity

Dentro de MainActivity, en el bloque setContent, mostrar PantallaSincronizacion:



Figura 10: Código (MainActivity.kt completo)

Paso 11. Ejecución en emulador y verificación en Logcat

Ejecutar la aplicación en un emulador, pulsar «Sincronizar» una vez y verificar en Logcat (filtrando con tag:SYNC) que aparece el mensaje «1. Servicio iniciado», mientras la UI muestra el indicador de progreso.

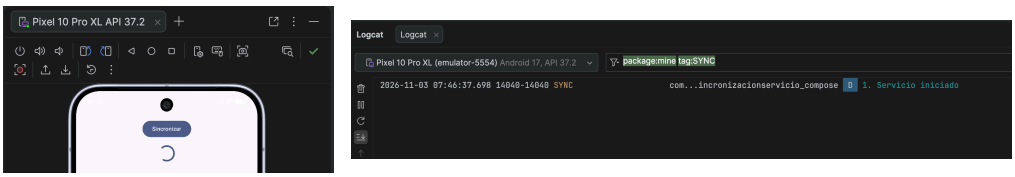


Figura 11: Paso 11: Ejecución de la aplicación en el emulador con el indicador de progreso activo y verificación en Logcat («1. Servicio iniciado»).

Paso 12. Confirmar sincronización y recepción del Broadcast

Esperar a que termine la coroutine (unos 3 segundos) y confirmar que en Logcat aparecen «2. Aviso enviado» y «3. Aviso recibido en la pantalla», y que la UI reemplaza el indicador por el mensaje recibido mediante el BroadcastReceiver.

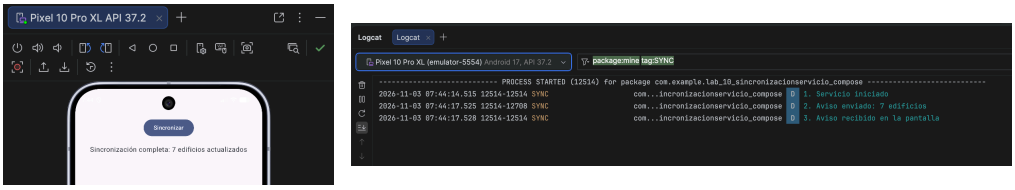


Figura 12: Paso 12: Confirmación en Logcat de la finalización de la coroutine y actualización reactiva de la UI mediante el BroadcastReceiver.

EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el ejercicio resuelto para dar más control al usuario sobre la sincronización:

1. Agrega un botón «Detener sincronización» que llame a context.stopService(Intent(context, SincronizacionService::class.java)) mientras la tarea está en curso.

Para dotar al usuario de control para detener la sincronización en ejecución, se extendió la interfaz en `PantallaSincronizacion()` dentro de `MainActivity.kt`.

Cambios Realizados

- **Archivo modificado:** `app/src/main/java/com/example/lab_10_sincronizacionservicio_compose/MainActivity.kt`

1. **Nuevo estado `enCurso`:** Se incorporó `var enCurso by remember { mutableStateOf(false) }` para modelar de forma reactiva si el servicio de sincronización está activo.
2. **Inicio de sincronización:** Al pulsar **«Sincronizar»**, se reinicia `resultado = null`, se asigna `enCurso = true` y se invoca `context.startService(...)`.
3. **Finalización por Broadcast:** En el `BroadcastReceiver`, al recibir la confirmación de término de sincronización, se asigna `enCurso = false` y se actualiza el mensaje con los edificios procesados.
4. **Botón «Detener sincronización»:** Se añadió un botón condicional visible únicamente mientras `enCurso == true`. Al hacer clic, invoca `context.stopService(Intent(context, SincronizacionService::class.java))` y actualiza el estado local a `enCurso = false`.
5. **Indicador visual reactivo:** El componente `CircularProgressIndicator()` se gobierna ahora según el valor de `enCurso` (en lugar de evaluar `resultado == null`).
6. **Alcance de modificaciones:** No se requirió alterar el servicio `SincronizacionService`, el archivo `AndroidManifest.xml` ni las importaciones previas.

Código Implementado en `PantallaSincronizacion()`

```
// 1. Estado reactivo para rastrear si la sincronización está activa
var enCurso by remember { mutableStateOf(false) }

// En el BroadcastReceiver:
```

```kotlin
val receiver = object : BroadcastReceiver() {
    override fun onReceive(c: Context?, intent: Intent?) {
        val cantidad = intent?.getIntExtra("CANTIDAD", 0) ?: 0
        resultado = "Sincronización completa: $cantidad edificios actualizados"
        // 3. Se desactiva el estado al recibir el aviso de finalización
        enCurso = false
    }
}

// Dentro del Column:
Button(onClick = {
    resultado = null
    // 2. Se activa el estado al iniciar el servicio
    enCurso = true
    context.startService(Intent(context, SincronizacionService::class.java))
}) {
    Text("Sincronizar")
}

// 4. Botón visible únicamente mientras la tarea está en curso
if (enCurso) {
    Spacer(modifier = Modifier.height(16.dp))
    Button(onClick = {
        context.stopService(Intent(context, SincronizacionService::class.java))
        enCurso = false
    }) {
        Text("Detener sincronización")
    }
}

Spacer(modifier = Modifier.height(16.dp))

// 5. El indicador de carga o el resultado se renderizan según enCurso
if (enCurso) {
    CircularProgressIndicator()
} else {
    Text(resultado ?: "")
}
```

Evidencias

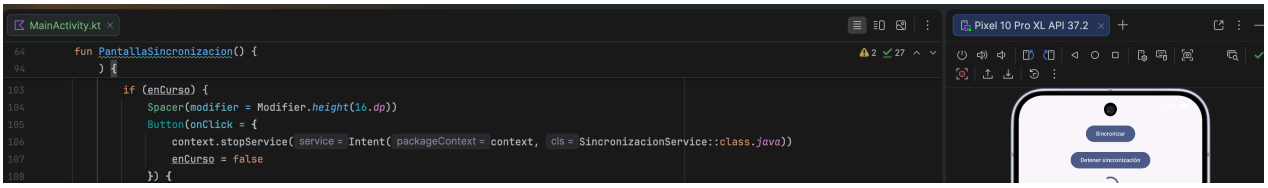


Figura 13: Implementación en `MainActivity.kt` y visualización simultánea de la interfaz en Android Studio.

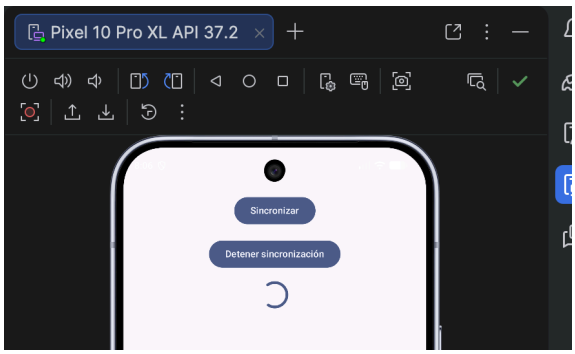


Figura 14: Emulador ejecutando la sincronización, mostrando el botón «Detener sincronización» y el indicador de progreso activo.

2. Modifica el Service para que, en lugar de un resultado simulado, recorra la lista de edificios persistida en Room (Laboratorio 09) y «sincronice» (por ejemplo, marque como actualizados) los que cumplan una condición, enviando el conteo real por el broadcast.

Se sustituyó la sincronización simulada por una persistencia real con Room (reutilizando el esquema del Laboratorio 09), sincronizando únicamente aquellos edificios con `sincronizado == false` y reportando métricas reales a la UI.

Archivos Configurados y Modificados

Archivo	Cambio Realizado
`EdificiosRoom.kt`	Nueva entidad `EdificioEntity` (con flags `sincronizado` y `actualizadoEn`), `EdificioDao`, `AppDatabase`, `DatabaseProvider` y función de sembrado inicial.
`MainActivity.kt`	El servicio consulta Room, actualiza pendientes en bucle con delay, cancela su `Job` en `onDestroy()` y emite el conteo real; la UI muestra «N de M».
`build.gradle.kts` / `app`	Configuración de KSP (`2.3.6`) y dependencias de Room (`2.7.2`: runtime, ktx y compiler).
`libs.versions.toml`	Declaración de dependencias y versiones de Room y KSP en el catálogo de versiones.

1. Estructura de Room (`EdificiosRoom.kt`)

Se definió la entidad y el DAO para gestionar la condición de sincronización:

```kotlin
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val categoria: String,
    val sincronizado: Boolean = false,   // <-- estado de sincronización
    val actualizadoEn: Long? = null
)

@Dao
interface EdificioDao {
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

```

```
    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    // Condición: edificios pendientes de sincronizar
    @Query("SELECT * FROM EdificioEntity WHERE sincronizado = 0")
    suspend fun obtenerPendientes(): List<EdificioEntity>

    @Query("UPDATE EdificioEntity SET sincronizado = 1, actualizadoEn = :momento WHERE id = :id")
    suspend fun marcarSincronizado(id: Int, momento: Long)
}
```

Incluye además `AppDatabase`, `DatabaseProvider` (siguiendo el patrón del Lab 09) y la función `sembrarEdificiosIniciales(dao)`, la cual inserta 4 edificios de ejemplo (los 2 del Lab 09 más Pabellón de Aulas y Comedor) únicamente si la base está vacía, asegurando datos de prueba reproducibles.

2. Consulta Real y Cancelación en `SincronizacionService`

El servicio consulta Room en un hilo IO, actualiza cada registro pendiente y vincula la corrutina a un `Job` para corte inmediato al detenerse:

```
private var trabajo: Job? = null

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    trabajo = CoroutineScope(Dispatchers.IO).launch {
        val dao = DatabaseProvider.obtener(applicationContext).edificioDao()
        sembrarEdificiosIniciales(dao)

        val pendientes = dao.obtenerPendientes()   // Condición real
        var actualizados = 0

        for (edificio in pendientes) {
            delay(500) // Simula la sincronización remota
            dao.marcarSincronizado(edificio.id, System.currentTimeMillis())
            actualizados++
        }

        val resultado =
Intent("com.example.lab_10_sincronizacionservicio_compose.ACCION_SYNC_LISTA").apply {
            setPackage(packageName)
            putExtra("edificios_actualizados", actualizados) // Conteo real
            putExtra("edificios_revisados", pendientes.size)
        }
        sendBroadcast(resultado)
        stopSelf()
    }
    return START_NOT_STICKY
}

override fun onDestroy() {
    trabajo?.cancel()   // "Detener sincronización" cancela la corrutina activamente
    super.onDestroy()
}
```

3. Actualización de la UI con Conteo Real

En el `BroadcastReceiver` de `PantallaSincronizacion`, se procesan los datos reales reportados por el servicio:

```kotlin
val revisados = intent?.getIntExtra("edificios_revisados", 0) ?: 0
val cantidad = intent?.getIntExtra("edificios_actualizados", 0) ?: 0

resultado = if (revisados == 0) {
    "Sin edificios pendientes de sincronizar"
} else {
    "Sincronización completa: $cantidad de $revisados edificios actualizados"
}
```

Evidencias

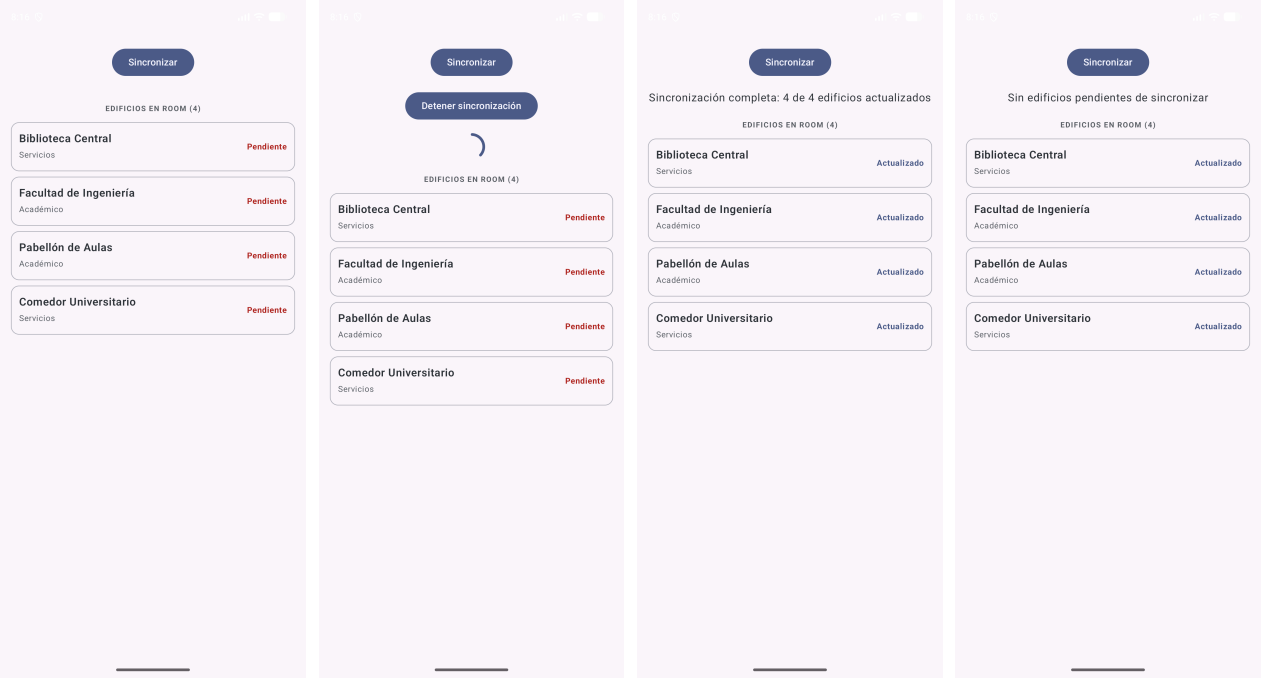


Figura 15: Ciclo de vida y sincronización real con Room: (a) 4 edificios pendientes, (b) sincronización en curso con indicador y botón de detención, (c) primera sincronización (4 de 4 actualizados), y (d) segunda sincronización sin registros pendientes.

3. Registra un segundo BroadcastReceiver que escuche una acción distinta (por ejemplo «com.tuapp.ACCION_SYNC_ERROR») y simula un caso de error en el Service (por ejemplo, con una probabilidad aleatoria), mostrando un mensaje distinto en la UI.

Se introdujo la acción de difusión `ACCION_SYNC_ERROR` para simular y canalizar fallos de sincronización. Antes de consultar Room, el servicio evalúa una probabilidad del 30 % (`Random.nextInt(100) < 30`); de cumplirse, emite el broadcast con el extra `mensaje_error`, escribe en Logcat y ejecuta `stopSelf()` sin modificar la base de datos. En la UI, un segundo `BroadcastReceiver` captura la señal, apaga el indicador de carga y muestra el mensaje de error.

Cambios Realizados

- **Archivo modificado:** `app/.../MainActivity.kt`

1. **Acción adicional:** Definición de `ACCION_SYNC_ERROR` junto a `ACCION_SYNC_LISTA`.
2. **Fallo en segundo plano (30 %):** Al dispararse, envía el broadcast con `mensaje_error` y sale con `stopSelf()`.
3. **Segundo receptor reactivo:** `receiverError` (`RECEIVER_NOT_EXPORTED`) actualiza `resultado` y apaga `enCurso`.
4. **Ciclo de vida y consistencia:** Ambos receptores se desregistran en `onDispose` y los 4 edificios siguen **Pendiente**.

Código Implementado

```
// 1. Acciones de difusión
private const val ACCION_SYNC_LISTA =
"com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA"
private const val ACCION_SYNC_ERROR =
"com.example.sincronizacionservicio_compose.ACCION_SYNC_ERROR"
// 2. Simulación de fallo aleatorio (30%) en el Service
delay(800)
if (Random.nextInt(100) < 30) {
    val error = Intent(ACCION_SYNC_ERROR).apply {
        setPackage(packageName)
        putExtra("mensaje_error", "No se pudo conectar con el servidor de sincronización")
    }
    sendBroadcast(error)
    Log.d("SYNC", "2b. Error de sincronización notificado")
    stopSelf()
    return@launch
}
// 3. Segundo BroadcastReceiver en PantallaSincronizacion
val receiverError = object : BroadcastReceiver() {
    override fun onReceive(ctx: Context?, intent: Intent?) {
        Log.d("SYNC", "3b. Error recibido en la pantalla")
        val mensaje = intent?.getStringExtra("mensaje_error") ?: "Error desconocido"
        resultado = "Error de sincronización: $mensaje"
        enCurso = false
    }
}
ContextCompat.registerReceiver(
    context, receiverError, IntentFilter(ACCION_SYNC_ERROR),
    ContextCompat.RECEIVER_NOT_EXPORTED
)
// 4. Desregistro de ambos receptores en onDispose
onDispose {
    context.unregisterReceiver(receiver)
    context.unregisterReceiver(receiverError)
}
```

Evidencias de Ejecución

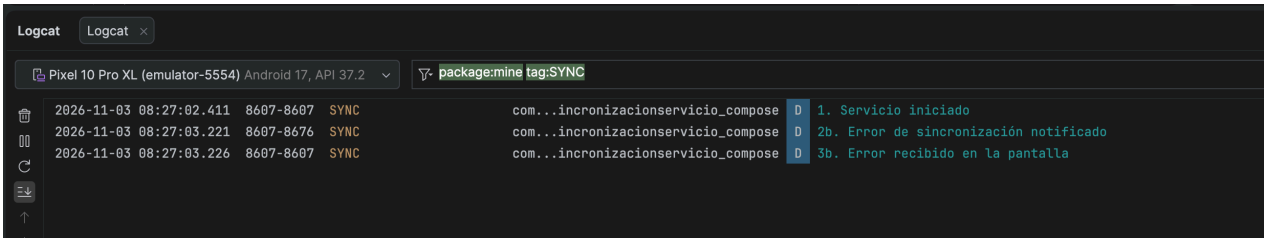


Figura 16: Registro en Logcat mostrando la secuencia de fallo: servicio iniciado (`1`), notificación de error desde el servicio (`2b`) y recepción en la pantalla (`3b`).



Figura 17: Emulador reflejando el mensaje de error capturado por el segundo receptor; nótese que los edificios permanecen en estado «Pendiente».

4. Investiga qué son los WorkManager de Android y explica en el informe en qué casos convendría usar WorkManager en lugar de un Service iniciado manualmente para esta misma tarea de sincronización.

WorkManager (`androidx.work`) es la biblioteca de Android Jetpack para ejecutar **trabajo en segundo plano diferible y garantizado**. A diferencia de un `Service` manual (que corre en la memoria RAM y muere al cerrarse la app), WorkManager persiste las tareas en SQLite interno, reprogramándolas incluso tras reiniciar el dispositivo o bajo políticas de ahorro energético (**Doze Mode**).

Criterio	Service Manual (`startService`)	WorkManager (`androidx.work`)
Disparo y Momento	Manual e inmediato desde la UI (tiempo real)	Diferible; programado según condiciones del sistema
Persistencia / Reinicio	Efímero; se pierde al cerrar la app o reiniciar	Garantizado; persiste en SQLite interno tras reinicios
Restricciones	Verificación manual imperativa en código	Declarativas (`Constraints`: red Wi-Fi, batería, carga)
Gestión de Fallos	Manual; requiere lógica de reintento propia	Reintentos automáticos con política de **backoff**
Propósito Idóneo	Interacción en pantalla con respuesta inmediata	Sincronización periódica o diferida desatendida

Criterio de decisión para la sincronización de edificios (Lab 10):

- **¿Cuándo convendría WorkManager?** Si Room debe actualizarse de forma periódica (p. ej., cada 12 h sin abrir la app), con red Wi-Fi obligatoria (`NetworkType.UNMETERED`), reintentos automáticos con **backoff** y garantía de ejecución tras reinicios.

- **¿Por qué se mantiene el Service en el laboratorio?** Porque la tarea es interactiva: el usuario pulsa **«Sincronizar»** esperando feedback visual instantáneo (spinner, progreso y botón **«Detener»**). WorkManager es diferible y no garantiza ejecución inmediata en la UI.



Figura 18: Service manual (ejecución efímera en RAM) vs. WorkManager (persistencia en SQLite con ejecución condicionada).

Implementación de referencia con Room:

```
// Worker con Room, reintentos con backoff y programación periódica con restricciones
class SyncWorker(ctx: Context, p: WorkerParameters) : CoroutineWorker(ctx, p) {
    override suspend fun doWork(): Result = try {
        val dao = DatabaseProvider.obtener(applicationContext).edificioDao()
        dao.obtenerPendientes().forEach { dao.marcarSincronizado(it.id,
System.currentTimeMillis()) }
        Result.success()
    } catch (e: Exception) { Result.retry() }
}
val req = PeriodicWorkRequestBuilder<SyncWorker>(6, TimeUnit.HOURS)
   .setConstraints(Constraints.Builder().setRequiredNetworkType(NetworkType.CONNECTED).build()).build()
WorkManager.getInstance(context).enqueueUniquePeriodicWork("sync_edificios",
ExistingPeriodicWorkPolicy.KEEP, req)
```

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (servicio en segundo plano que sincroniza datos y notifica el resultado mediante un BroadcastReceiver)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, logré resolver completamente el ejercicio resuelto del docente siguiendo cada uno de los pasos indicados. Al desarrollarlo pude entender en la práctica cómo conectar un servicio en segundo plano con una interfaz moderna en Jetpack Compose utilizando un BroadcastReceiver.

A continuación, adjunto las capturas del emulador y Logcat que evidencian el funcionamiento exitoso de la solución:

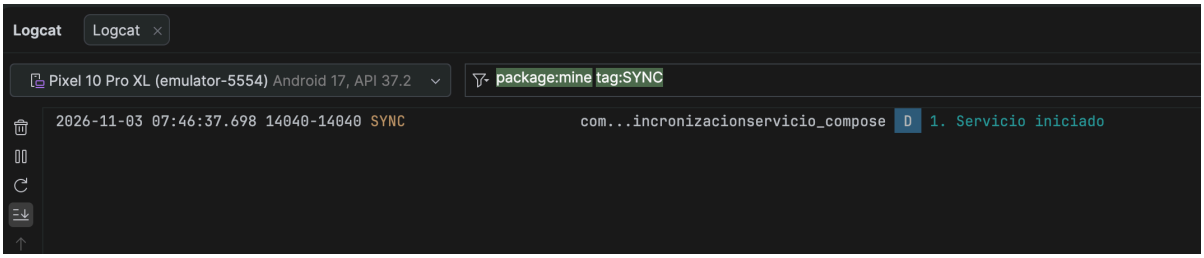


Figura 19: Evidencia 1: Momento en que presiono el botón «Sincronizar»; el emulador muestra la barra de progreso activa y en Logcat se registra «1. Servicio iniciado».

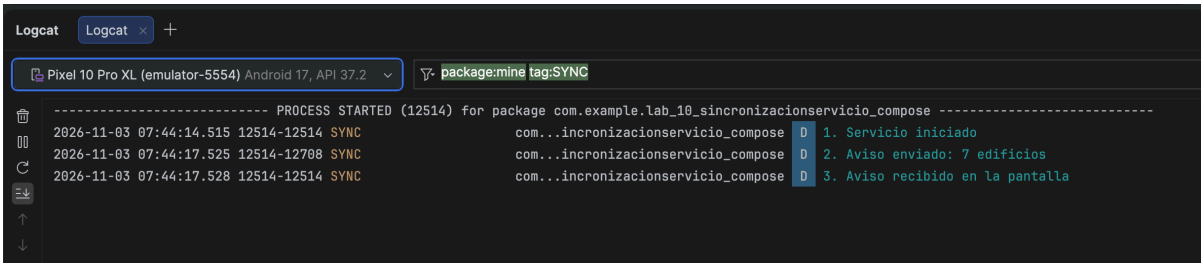


Figura 20: Evidencia 2: Luego de los 3 segundos, Logcat muestra «2. Aviso enviado» y «3. Aviso recibido en la pantalla», y la interfaz reemplaza la carga mostrando la cantidad de edificios sincronizados.

2. En tu propia implementación, ¿qué ocurriría si la tarea dentro de onStartCommand() se ejecutara directamente en el hilo principal en lugar de una coroutine en Dispatchers.IO? Explica el efecto sobre la interfaz de usuario.

Si en mi propia implementación hubiera puesto la tarea pesada o bloqueante directamente dentro de `onStartCommand()` (por ejemplo, usando un `Thread.sleep(3000)` en vez de la corrutina), en la interfaz ocurriría lo siguiente:

1. **La pantalla se congelaría por completo:** El hilo principal se encarga de refrescar la pantalla a 60 fps (cada 16 ms) y de hacer girar las animaciones. Si lo mantengo ocupado durante 3 segundos, la interfaz no podría dibujar nuevos fotogramas. En mi app, el circulito de carga (`CircularProgressIndicator`) no se movería en absoluto; se quedaría totalmente congelado o ni siquiera llegaría a renderizarse tras presionar el botón.
2. **La aplicación dejaría de responder al tacto:** Cualquier intento de presionar otro botón, hacer scroll o interactuar con la pantalla durante esos 3 segundos sería ignorado, dando la molesta sensación de que la app «se colgó» o se trabó.

III. CONCLUSIONES

- **Desmitificación del Service y la necesidad de corrutinas:** Uno de los aprendizajes más reveladores de la práctica fue comprobar que un `Service` en Android no crea por sí solo un hilo secundario, sino que se ejecuta en el mismo hilo principal de la aplicación. Al principio es fácil asumir que por llamarse «servicio en segundo plano» ya no afectará la interfaz; sin embargo, entender que corre en el Main Thread me dejó muy claro por qué es indispensable combinarlo con corrutinas en `Dispatchers.IO` para no congelar la pantalla, evitar caídas de fotogramas y prevenir el error ANR.
- **Desacoplamiento limpio mediante BroadcastReceiver:** Implementar un `BroadcastReceiver` como puente de comunicación entre un componente no visual (`Service`) y la pantalla en Compose me demostró el valor de una arquitectura desacoplada. El servicio no necesita conocer qué pantalla está abierta ni retener referencias a la interfaz (lo que causaría fugas de memoria); simplemente emite un mensaje con un `Intent` y la interfaz, al estar suscrita, reacciona de forma natural cuando llega la información.
- **Gestión responsable del ciclo de vida y recursos con `stopSelf()`:** Comprendí la importancia de que un servicio iniciado sea consciente de su propio ciclo de vida y se detenga apenas cumpla su tarea llamando a `stopSelf()`. Dejar servicios ejecutándose indefinidamente consume batería y memoria innecesaria, exponiendo la aplicación a que el sistema operativo la cierre de manera abrupta. Un buen diseño en Android debe cuidar los recursos del dispositivo tanto como la experiencia visual.
- **Seguridad moderna y ciclo de vida reactivo en Compose:** El uso de `DisposableEffect` junto a la directiva `RECEIVER_NOT_EXPORTED` me enseñó cómo convive el modelo declarativo de Jetpack Compose con las exigencias de seguridad de Android 14+. No solo aprendí a registrar y desregistrar el receptor automáticamente para no dejar escuchas activas al cambiar de pantalla, sino también a proteger la aplicación asegurando que ninguna otra app externa pueda interceptar o inyectar eventos de sincronización no autorizados.

Recomendaciones

- **Evaluar WorkManager para tareas de fondo persistentes:** Aunque un `Started Service` es adecuado para tareas inmediatas que el usuario inicia conscientemente durante la sesión, si en un futuro la sincronización debe completarse incluso si el usuario cierra la aplicación o el dispositivo se reinicia, recomiendo migrar o integrar `WorkManager`. Esta API se encarga de manejar automáticamente las restricciones del sistema operativo, el estado de la batería y la conectividad sin riesgo de ser destruida en versiones modernas de Android.
- **Implementar cancelación cooperativa mediante Jobs en corrutinas:** Sugiero asociar la corrutina de sincronización en `Dispatchers.IO` a un `Job` almacenable dentro del servicio. De esta forma, si el usuario decide cancelar la sincronización mediante un botón de «Detener» o si se invoca `stopService()`, se puede llamar a `job?.cancel()`, deteniendo de inmediato la tarea de red o cálculo en curso y evitando procesar o emitir broadcasts innecesarios.
- **Evolucionar hacia StateFlow y ViewModel para comunicación interna:** Si la comunicación de datos se limita exclusivamente al interior de la propia aplicación y no requiere comunicarse con otras aplicaciones o procesos del sistema, recomiendo considerar el uso de flujos reactivos (`StateFlow` o `SharedFlow`) alojados en un `ViewModel` o repositorio compartido. Esto simplifica las pruebas unitarias, elimina la serialización manual en los extras del `Intent` y proporciona tipado estricto en tiempo de compilación.
- **Comprobar la conectividad previa y enriquecer los estados visuales:** Recomiendo validar el estado real de la red (mediante `ConnectivityManager`) antes de disparar el servicio, avisando oportunamente al usuario si no hay conexión antes de iniciar un proceso condenado a fallar. Asimismo, es una buena práctica enriquecer la UI mostrando estados de progreso intermedios (como porcentajes o etapas de la sincronización) y mensajes amigables en caso de error, brindándole total transparencia y tranquilidad al usuario sobre lo que ocurre de fondo.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.